Performance test strategy

lendly · Static web build served from GitHub Pages (https://kranmal.github.io/lendly/) and a local copy of dist/ · 2026-10-22 · analysed from codebase

> **Status: proposal for approval.** No performance tests have been written or run. Answer the open questions and approve (or amend) the test plan before the build phase starts.

1. Summary

Lendly has no backend. It is an Expo/React Native app exported as a static web site; all data lives in the browser (AsyncStorage, which is localStorage on web). A classic server load test (hundreds of virtual users) would only hammer GitHub Pages, which is not ours to load and which scales on its own. The real performance risks are on the client: a 1.1 MB (314 KB gzipped) single JavaScript bundle on first load, and data-handling code that gets slower as a user's lists grow (every list item recomputes all active loans, nothing is virtualised, and every change rewrites all stored data). The proposed strategy therefore measures (1) first-load speed and delivery, (2) client performance at 100 / 1,000 / 5,000 / 20,000 records, and (3) a small, polite k6 check of static delivery against a local copy only. Traffic-style load, stress and spike tests against the live site are explicitly out of scope.

2. What the system is

Aspect	Detail
stack	Expo SDK 57, React Native 0.86 + react-native-web, expo-router (static output), React 19 with React Compiler
architecture	Client-only SPA, 3 prerendered routes (/, /items, /people) hydrating into one JS bundle. State is a module-level store with listeners (src/lib/store.ts).
dataStores	Browser localStorage via AsyncStorage: keys lendly:items, lendly:people, lendly:loans. No server database.
externalDeps	Google AdSense script and ad slot (src/components/ad-unit.tsx); GitHub Pages hosting and CDN (Fastly); deployed by .github/workflows/deploy-pages.yml
notes	Mobile (iOS/Android) builds exist via Expo; this strategy covers the web build because that is what is deployed. Native runtime performance would need device profiling and is not covered.

3. Findings that drive the strategy

Area	Observation	Evidence	Perf risk
First load	One JS entry bundle of 1.1 MB raw, 314 KB gzipped (measured from dist/). No route-level splitting. Total dist is 1.3 MB.	`dist/_expo/static/js/web/entry-*.js`	medium

Delivery	Live site responds in 156 to 286 ms (TTFB, 3 sequential requests, Frankfurt edge), HTTP/2, gzip. HTML cache-control is max-age=600, so a returning visitor revalidates every 10 minutes. Cache headers for the hashed JS are not controllable on GitHub Pages.	`curl -I https://kranmal.github.io/lendly/ (2026-10-22)`	low
Third party	AdSense script loads on every page and injects an ad slot; it adds network, main-thread cost and layout shift that we do not control. It must be excluded or isolated when measuring our own code.	`src/components/ad-unit.tsx:15-23`	medium
Items list	For every item, getLoanForItem calls getActiveLoans, which filters, enriches (linear item and person lookups per loan) and sorts ALL loans. Cost per render grows roughly with items x loans x (items + people) and runs twice on the Items screen (filter and row render).	`src/lib/store.ts:172-179 and src/app/items.tsx:22-24, 47-48`	high
List rendering	All lists use ScrollView with .map, so every row is mounted at once; there is no FlatList or virtualisation.	`src/app/index.tsx:65, src/app/items.tsx:47, src/app/people.tsx:40`	medium
Persistence	Every mutation rewrites all three localStorage keys with JSON.stringify of the full arrays, synchronously on the main thread (AsyncStorage on web). Cost is linear in total data size per tap.	`src/lib/store.ts:36-42, 31-34`	medium
Startup	On load the app reads and JSON.parses all three keys before the first meaningful render (loaded flag). A user with years of history pays this on every visit. localStorage is limited to about 5 MB per origin, which is a hard ceiling to test toward.	`src/lib/store.ts:44-58`	medium
Re-render	useNow ticks every 60 s and every mounted LoanRow subscribes, so each tick re-renders all visible rows. useStore re-renders every subscriber on any change.	`src/hooks/use-now.ts, src/lib/store.ts:60-70`	low
No server	There are no API endpoints, database, auth or queues to load test.	`package.json dependencies; src/ contains no fetch or API calls`	low

4. User journeys to test

Journey	Steps	Traffic share	Criticality	Endpoints

First visit (cold)	1. Open /
	2. Download HTML, bundle, assets
	3. Hydrate, read localStorage, first meaningful render
		unknown (assumed 60% of visits)	high	`GET /lendly/`
				`GET /lendly/_expo/static/js/web/entry-*.js`
				`GET /lendly/assets/*`

Returning visit with data	1. Open / 2. Load and parse stored data 3. Render active loans	unknown (assumed 40%)	high	`same as above + localStorage reads`
Browse items and people	1. Tap Items tab 2. Tap People tab 3. Open an item detail	n/a (client only)	medium	`client route changes`
Record a loan	1. Open Lend modal 2. Pick item 3. Enter person 4. Save	n/a (client only)	high	`store.lendItem + persist`
Mark returned	1. Open loan 2. Tap Returned	n/a (client only)	medium	`store.markReturned + persist`

5. Workload model

Concurrent users: **not applicable: the server work per user is a static file fetch** · Peak requests/s: **unknown** · Basis: No analytics available to the analysis. Dataset sizes below are assumptions about plausible personal use.

- Typical user: 10 to 100 items, 5 to 30 people, 20 to 300 loans (assumed).
- Heavy user: 1,000 items, 100 people, 5,000 loans (assumed upper realistic bound).
- Extreme test point: 20,000 loans to find where the app breaks or localStorage fills (stress equivalent).
- Test device: desktop Chromium with 4x CPU throttling to approximate a mid-range phone (assumed), plus unthrottled.

6. Service level objectives (pass/fail criteria)

Metric	Target	Rationale

Largest Contentful Paint, cold load, 4x CPU throttle, simulated 4G	< 2.5 s	Google 'good' threshold; also affects search ranking, which the SEO work targets.
Total Blocking Time, cold load	< 300 ms	Lighthouse 'good' threshold; reflects the 1.1 MB bundle parse cost.
Cumulative Layout Shift	< 0.1	Ad slot injection is the likely source.
Time to interactive list at 100 / 1,000 loans	< 500 ms / < 1.5 s	Assumed usability limits; to be confirmed by you.
Add-loan action (tap to UI updated), at 1,000 loans	< 100 ms	Common threshold for instant feel.
Items tab render time at 1,000 items and 1,000 loans	< 1 s	Highest-risk code path (finding 4).
Static delivery from local dist: p95 latency, error rate	p95 < 100 ms, errors < 1%	Sanity check of asset delivery with k6 against a local server only.

7. Test plan

Test	Purpose	Load	Duration	Pass criteria	Why this test
smoke	Prove the harness works: serve dist locally, load the three routes, k6 fetches them once.	1 virtual user	30 s	All routes 200, assets fetched	Required first step for every later test.
Cold-load audit (Lighthouse or Playwright tracing)	Measure LCP, TBT, CLS, bundle parse time, with and without the AdSense script.	Single page loads, 10 repeats, throttled and unthrottled	about 10 min	Web-vitals SLOs above	This is where the 1.1 MB bundle shows up; k6 cannot see render cost.
Data-scale test (Playwright, seeded localStorage)	Seed 100, 1,000, 5,000, 20,000 loans (with proportional items and people), then measure startup, tab render time, add-loan latency and mark-returned latency using the Performance API.	4 dataset sizes, 10 repeats each	about 20 min	Client SLOs above; report the dataset size where each SLO is first broken	This is the realistic stress test for a client-only app and targets findings 4 to 6.
load (k6, local dist only)	Check static delivery of dist under concurrent fetches, e.g. 50 virtual users for 2 minutes.	50 VUs (local server)	3 min	p95 < 100 ms, errors < 1%	Cheap sanity check; note it measures the local static server, not GitHub Pages, so it says little about production.

Storage limit probe	Find how many records fit before localStorage throws and what the app does when it does.	Single browser, growing dataset	5 min	App degrades gracefully (error handled, no data loss shown)	Persist has no error handling for quota errors (src/lib/store.ts:36).
Soak (optional)	Leave the app open for 1 hour with 1,000 loans and watch memory and the 60 s tick re-renders.	1 browser tab	1 h	No JS heap growth trend	Low risk; skip unless you want it.

8. Environment, data and safeguards

Aspect	Detail
Target	Local only: `npx expo export -p web` then a local static server on localhost, driven by Playwright (already in e2e-tests/) and k6. Live site: at most single read-only requests (as done during this analysis).
Test data	Synthetic generated items, people and loans injected into localStorage. No real data exists server-side.
Safeguards	• No load test against kranmal.github.io, GitHub Pages or AdSense (not ours, terms of service, no benefit). • k6 host guard allows localhost only. • AdSense is blocked in measurement runs unless the run is explicitly the 'with ads' variant. • No changes to the lendly source; findings only. The perf/ folder is added uncommitted.
Monitoring	• Chromium Performance API marks, long-task entries and JS heap size • Lighthouse metrics • k6 metrics for the local delivery check

9. Tooling

Playwright (already used in e2e-tests/) plus Lighthouse for the browser measurements, and k6 only for the local static-delivery check: The risks are in the browser, which k6 does not run. Playwright can seed localStorage, drive the UI and read performance entries. k6 adds little here but keeps the same harness and report as other projects.
Alternatives considered: JMeter or Locust would only test the server and add nothing for a client-only app. k6 browser module is possible but less mature than Playwright for this.

10. Risks

Risk	Mitigation
Laptop CPU noise makes timings vary	10 repeats, report median and p95, close other apps, same machine for before/after comparisons.
CPU throttling is only an approximation of a real phone	State this in the report; optionally confirm on a real device later.

Dataset assumptions may not match real use	Report results as a curve over dataset size, so you can read off your own number.
Local dist differs from the live deployment (base path /lendly, CDN)	Build with GH_PAGES_BASE_URL=/lendly as the deploy workflow does.

11. Open questions for you

- Do you have real usage numbers (visitors per day, device mix, typical number of items and loans)? Without them the dataset sizes are assumptions.
- Which matters more: web (SEO, first load) or the mobile apps? This plan covers web only.
- Are you happy to measure with the AdSense ad blocked by default, plus one 'with ads' run?
- Are the proposed SLOs (LCP < 2.5 s, add-loan < 100 ms, Items tab < 1 s at 1,000 records) the right bar?
- Is it OK for me to add the perf/ folder to the lendly repo (uncommitted, you decide whether to commit via a PR)?

12. Not in scope

- Load, stress or spike testing GitHub Pages or any live third-party service
- Native iOS and Android runtime performance
- Fixing the performance issues (findings are reported, not changed)
- Server capacity planning: there is no server

13. What happens after approval

1. Scripts and `perf.config.json` are built to this plan.
2. Smoke test, then load, stress and spike run one at a time.
3. A results report PDF is produced: SLO pass/fail, latency percentiles, throughput, error rates, breaking point, bottleneck notes and recommendations.